

VEK 0.4 / Wekretia

Game Development Project Documentation

An early-stage 2D RPG prototype developed in Godot Engine using
GDScript,
set in the original dark fantasy universe of Wekretia.

Author

Daniel Abramek

Main technologies

Godot Engine, GDScript, Krita, Aseprite, Git

Contents

1	Project Overview	3
1.1	What VEK 0.4 Is	3
1.2	Current Development Stage	3
2	Background of the Wekretia Universe	3
2.1	Origin of the World	3
2.2	General Character of the Setting	4
2.3	Regions and Cultures	4
3	Evolution of the Game Concept	4
3.1	Earlier Technical Approach	4
3.2	Early Gameplay Direction	5
3.3	Current Direction	5
4	Technology Stack	6
5	Current Prototype Architecture	6
5.1	General Structure	6
5.2	Simplified Prototype Structure	6
6	Player Controller	6
6.1	Player Node	6
6.2	Input Actions	7
7	Movement System	7
7.1	Basic Movement	7
7.2	Last Direction Tracking	7
8	Animation System	8
8.1	Movement and Idle Animations	8
8.2	Attack Animations	8
9	Basic Attack State	8
9.1	Attack State	8
9.2	Current Limitations	9
10	Development Method	9
10.1	Learning Through Iteration	9
10.2	Areas Being Learned	9
11	Visual Direction	10
12	Combat Design Direction	10
13	Interactive World Design	11
14	Dialogue and Consequence Design	11
15	World Scope	11

16 Implemented Functionality vs. Design Goals	12
16.1 Implemented	12
16.2 Planned or Experimental	12
17 Current Project Goals	13
18 Possible Future Development	13
19 Conclusion	14

1 Project Overview

1.1 What VEK 0.4 Is

VEK 0.4 is an early-stage 2D role-playing game prototype developed in Godot Engine using GDScript.

The project is set in Wekretia, an original universe that existed long before the current game project. The world has been developed over many years through worldbuilding, stories, roleplay, characters, religions, nations, conflicts, and local histories.

The game is therefore not based on a setting created only for the purpose of making a game. The development process works in the opposite direction: an existing fictional world is gradually being translated into an interactive medium.

1.2 Current Development Stage

The project is currently in pre-production and early prototyping.

The main goal at this stage is to build and understand the core systems one at a time before attempting to create the larger RPG structure.

The current implemented prototype focuses primarily on:

- player movement,
- directional animation,
- input handling,
- basic attack state logic,
- early project structure,
- initial UI and menu work.

Larger systems such as quests, dialogue, factions, advanced AI, inventory, equipment, and complex narrative consequences are still design goals rather than finished features.

2 Background of the Wekretia Universe

2.1 Origin of the World

Wekretia has been developed for many years.

Before the game project existed, the universe was used for worldbuilding and roleplay. Some events created by players during roleplay sessions became part of the world canon rather than being discarded after the session.

This means that Wekretia has developed both through authored worldbuilding and through stories that emerged organically from people interacting with the setting.

2.2 General Character of the Setting

Wekretia is a dark fantasy world with grotesque and partially industrial elements.

The setting deliberately avoids relying on a conventional fantasy structure based entirely on spellcasting and traditional magic systems.

One of the central elements of the world is the use of dangerous crystals. Depending on the region and application, crystals may function as:

- energy sources,
- foundations for technology,
- rare and valuable resources,
- dangerous materials associated with effects similar to radiation.

As a result, the setting exists somewhere between medieval society, dark fantasy, early industrialization, and strange crystal-based technology.

2.3 Regions and Cultures

Wekretia is not a single kingdom.

It is a larger world containing multiple nations, cultures, religions, races, political conflicts, and local histories.

The design goal is for regions to differ through more than terrain or visual palette. Each area should eventually communicate its own:

- history,
- architecture,
- culture,
- religion,
- political conflicts,
- way of life,
- relationship with neighbouring nations.

3 Evolution of the Game Concept

3.1 Earlier Technical Approach

Before the current Godot version, the project was briefly approached as a more general programming experiment using Java and JavaFX.

That approach was eventually abandoned.

The decision was made to learn and use an actual game engine rather than forcing a general-purpose GUI framework into a role it was not designed for.

Around April 2026, the project moved to:

Godot Engine + GDScript

3.2 Early Gameplay Direction

One of the earliest concrete ideas was a 2D action RPG inspired in part by games such as *Tails of Iron*.

The early concept included:

- real-time combat,
- exploration,
- character progression,
- story,
- dialogue,
- quests,
- a darker visual tone.

The project initially leaned more toward a side-view action game with platforming elements.

3.3 Current Direction

As the design evolved, it became clear that the amount of existing lore, factions, characters, dialogue, and history in Wekretia fits an RPG structure better than a traditional platformer.

The current direction is therefore a top-down 2D RPG, potentially with a slightly isometric appearance.

The intended focus is on:

- exploration,
- combat,
- immersion,
- interaction with the world,
- dialogue and narrative,
- player choices and consequences.

Atmospheric inspiration includes games such as *Darkwood*, although VEK is not intended to copy its mechanics or visual identity.

4 Technology Stack

Table 1: Main tools used in VEK 0.4

Technology	Role
Godot Engine	Main game engine and scene system.
GDScript	Gameplay and application scripting.
Krita	Source artwork and image editing.
Aseprite	Pixel-art and animation workflow.
Git / GitHub	Version control and public project history.

5 Current Prototype Architecture

5.1 General Structure

The current prototype is organized around Godot scenes and GDScript files.

At the current stage, the player is represented using `CharacterBody2D` with an `AnimatedSprite2D` used for character animation.

The project also contains early work related to menus, settings, UI, music, an intro scene, and an inn scene. These parts are still evolving and should not be treated as final game systems.

5.2 Simplified Prototype Structure

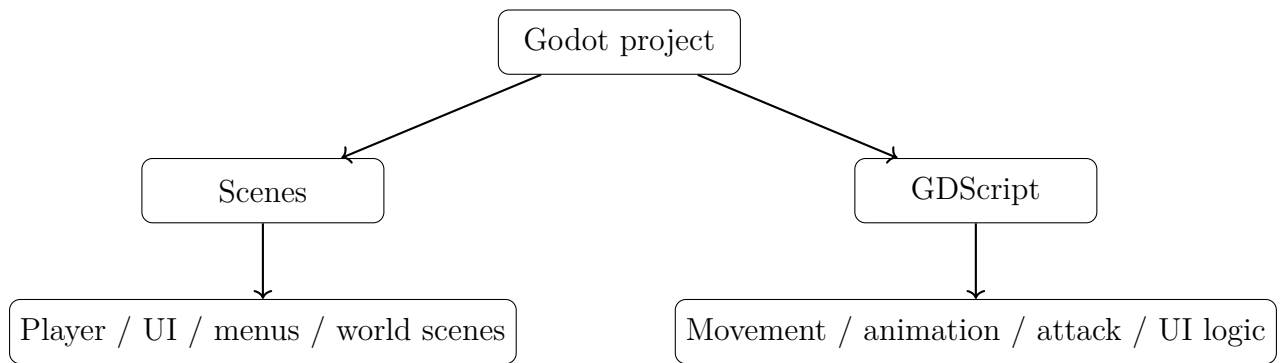


Figure 1: Simplified structure of the current prototype

6 Player Controller

6.1 Player Node

The first major gameplay system implemented in the Godot version was the player controller.

The player is based on:

`CharacterBody2D`

with:

used for the visual animation system.

6.2 Input Actions

Movement is controlled through custom Godot Input Map actions:

- `left_input`,
- `right_input`,
- `up_input`,
- `down_input`,
- `attack_input`.

Using named input actions instead of hard-coded keyboard checks makes the input layer easier to change later and allows future support for alternative controls.

7 Movement System

7.1 Basic Movement

The player can currently move in four directions.

The core movement logic is based on creating a direction vector and applying a speed value:

Listing 1: Simplified player movement

```
velocity = direction * SPEED
```

The early prototype used:

```
SPEED = 100.0
```

The exact value is not considered final and may change as the game feel develops.

7.2 Last Direction Tracking

The controller stores the most recent direction in which the player was facing.

Conceptually:

```
last_direction
```

This value is important because the correct idle and attack animations still need to be selected after the player stops moving.

8 Animation System

8.1 Movement and Idle Animations

The prototype includes directional movement and idle animation logic.

Movement animations include:

- `run_side`,
- `run_front`,
- `run_back`.

Idle animations include:

- `idle_side`,
- `idle_front`,
- `idle_back`.

For sideways movement, sprite flipping can be used instead of creating separate left-facing and right-facing animation sets.

8.2 Attack Animations

Directional attack animation work includes:

- `attack_side`,
- `attack_front`,
- `attack_back`.

This system is still part of the evolving prototype.

9 Basic Attack State

9.1 Attack State

The first combat-related implementation uses a dedicated attack state.

The prototype includes variables conceptually represented as:

```
is_attacking  
attack_timer
```

When the character is attacking, normal movement is blocked.

This prevents the player from moving freely while the attack animation is active and creates a foundation for a more complete combat state system.

9.2 Current Limitations

The current attack implementation is only an early step toward a real combat system.

The following systems should not be treated as completed:

- complete enemy hitboxes,
- advanced enemy AI,
- weapon system,
- equipment system,
- damage balancing,
- dodge mechanics,
- tactical mouse aiming.

10 Development Method

10.1 Learning Through Iteration

VEK 0.4 is both a game project and a learning project.

The current workflow is deliberately iterative:

1. build a small system,
2. test it,
3. debug problems,
4. clean up the implementation,
5. understand why it works,
6. move to the next system.

This approach is intended to avoid creating a large amount of fragile code before the underlying systems are understood.

10.2 Areas Being Learned

The project is used to develop practical experience with:

- Godot Engine,
- GDScript,
- gameplay programming,
- game-system design,
- 2D animation,

- collision handling,
- scene organization,
- level design,
- pixel-art workflow.

11 Visual Direction

The intended visual direction combines pixel-art foundations with a more hand-painted character.

The world should feel:

- dirty,
- dark,
- grotesque,
- organic,
- visually distinctive.

Different regions should eventually be recognizable through visual culture rather than only through text labels or biome changes.

12 Combat Design Direction

The long-term goal is for combat to be more deliberate than repeatedly pressing a single attack button while standing next to an enemy.

Ideas being considered include:

- real-time combat,
- directional attacks,
- mouse aiming,
- dodging,
- different weapon types,
- armour,
- environmental interactions.

These are design goals and are not all implemented in the current prototype.

The intended result is a relatively tactical combat system rather than a fast hack-and-slash structure.

13 Interactive World Design

One of the more ambitious design goals is to make the world feel physically and systemically interactive.

Ideas include:

- using the environment during combat,
- burning selected environmental objects,
- more physical interactions with objects,
- NPC reactions to player actions.

These concepts remain part of the long-term design direction and should not be treated as completed functionality.

14 Dialogue and Consequence Design

Player actions are intended to have consequences beyond a simple good-ending / bad-ending choice.

Possible consequences may eventually affect:

- relationships with characters,
- quests,
- political situations,
- available information,
- the fate of individual NPCs,
- how the player is perceived,
- parts of the world's evolving narrative.

This idea is connected to the history of Wekretia itself, where events created during roleplay were sometimes preserved as part of the canon.

15 World Scope

Despite the size of the Wekretia universe, the game is not intended to become a full simulation of the entire continent.

A more realistic design target is:

a large and rich world presented through a carefully directed journey across selected parts of it

This allows the project to preserve the scale and depth of the setting without requiring a single developer to simulate every region at once.

16 Implemented Functionality vs. Design Goals

A clear distinction is maintained between the current implementation and future design ideas.

16.1 Implemented

The following should be treated as actual development work completed or actively prototyped:

- Godot project setup,
- GDScript-based player controller,
- `CharacterBody2D` player,
- `AnimatedSprite2D`,
- custom Input Map,
- four-direction movement,
- direction tracking,
- movement and idle animation switching,
- early attack state,
- attack timer,
- movement blocking during attack,
- directional attack animation work,
- early UI and menu structure.

16.2 Planned or Experimental

The following should be treated as future design directions rather than finished features:

- large multi-region game world,
- complete quest system,
- complete dialogue system,
- factions and reputation,
- complex inventory,
- character progression,
- multiple weapon systems,
- advanced enemy AI,

- interactive environment,
- environmental fire,
- tactical mouse-aimed combat,
- large narrative consequence system,
- possible reincarnation mechanics,
- meta-narrative concepts involving the player as an outsider.

17 Current Project Goals

The short-term development goals are intentionally smaller than the full game vision.

The current priorities are:

1. improve the player controller,
2. continue building the combat foundation,
3. organize the project structure,
4. improve animation handling,
5. learn Godot and GDScript more deeply,
6. prototype additional gameplay systems only after the current ones are stable.

18 Possible Future Development

Future development may include:

- a reusable state machine for player states,
- proper hitbox and hurtbox systems,
- enemy prototypes,
- basic combat AI,
- inventory and equipment,
- dialogue data structures,
- quest data structures,
- save/load systems,
- region prototypes,
- improved level-design workflow,
- more complete UI,
- expanded animation tooling.

The exact order of implementation is expected to change as the prototype develops.

19 Conclusion

VEK 0.4 is an early-stage attempt to turn the long-running Wekretia universe into a real playable RPG.

The current project is deliberately modest on the implementation side. Instead of claiming large systems that do not yet exist, development is focused on building the foundations properly: movement, animation, input, state handling, combat basics, and project organization.

The long-term value of the project lies in combining two things: an existing fictional universe developed over many years and the practical process of learning game development through a real, evolving codebase.

The project remains a work in progress.